Agentic Graph Reasoning: Claim-Level Structural Verification for Multi-Hop Knowledge Graph Question Answering

Md. Sakif Khan[a], Sadia Sharmin[a]

[a]*Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*

Abstract

Multi-hop question answering over knowledge graphs requires composing facts across several edges, and systems that retrieve once before reasoning cannot know what the second hop will need. Agentic approaches interleave retrieval with reasoning, but none checks that what the final answer asserts is supported by what the agent actually retrieved. We present Agentic Graph Reasoning (AGR), a framework built as an explicit state machine over a constrained, deterministic graph tool API, in which a structural verification layer decomposes each draft answer into atomic claims and checks them against the traversed triples before emission, so every answer is returned with the evidence supporting it. On 400 certified questions from each of WebQSP and ComplexWebQuestions over a Freebase-derived environment, AGR reaches Hits@1 of 0.755 and 0.522 against four baselines sharing one frozen backbone and one call budget, using roughly half the language-model calls of the strongest. A component-level ablation with paired significance testing finds that removing the planner *improves* accuracy on the shallower

Email address: sakifkhanopu@gmail.com (Md. Sakif Khan)

benchmark while cutting tokens by 31%, and that three of four components — including claim verification — show no detectable effect on accuracy. A hand-read census of all 259 residual failures identifies the *echo attractor*, a true, grounded entity one hop from the answer that every grounding check passes and that consensus-based evaluation protocols cannot detect.
Keywords: knowledge graph question answering, large language models, agentic retrieval, ablation study, hallucination, benchmark quality

---

1. Introduction

Large language models answer factual questions fluently, and often enough they answer them wrongly. The errors are systematic rather than random, and they are hard to catch from the output alone, because a fabricated answer reads in the same register as a correct one; Huang et al. [1] treat this as an intrinsic consequence of how the models are built rather than an incidental defect to be patched away. The problem sharpens once an answer requires composing several facts rather than recalling one. Errors compound, since a wrong resolution at the first hop rarely announces itself downstream, and a question-level correctness check never sees the intermediate steps at all — which leaves a system that answers correctly by accident indistinguishable from one that answers correctly by sound reasoning.

Retrieval-augmented generation [2] addresses the first half of this by grounding the model in retrieved text. It does not address the second. Retrieval runs once, before reasoning begins, so a multi-hop question must be answered from whatever that single query returned, and the passage holding

the *intermediate* entity need not resemble the original question — which is the only signal the retriever has to rank on. Representing the corpus as a knowledge graph [3] makes composition a traversal rather than an inference over prose, and GraphRAG [4] exploits this, but it still retrieves statically: a neighbourhood fetched in one shot cannot know what the next reasoning step will need, and is either too small to contain the answer or so large that the relevant edges are buried.

Agentic systems close that gap by interleaving the two: the model takes one step, observes what it found, and decides where to look next [5, 6, 7]. Retrieval becomes navigation, and the system becomes an agent operating on the graph rather than a pipeline reading from it. Section 2 traces that line of work in detail.

One property survives this progression unchanged. Every system named above emits whatever its final generation call produces from the traversed context, and nothing checks that what the answer *asserts* is supported by what the agent actually retrieved. The gap is not fabrication — our measurements confirm that graph-navigating systems assert entities that exist in the graph and are reachable along the paths they walked (Section 5.4). It is that an entity can sit in the retrieved subgraph, have been retrieved correctly, and still be the wrong answer: an intermediate node from the reasoning chain, a container where the question asked for a member, or one of several plausible neighbours chosen arbitrarily. A system that verifies retrieval but never verifies assertion cannot catch either case.

This paper presents **Agentic Graph Reasoning** (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a

constrained, deterministic graph tool API, in which the program orchestrates the loop and the language model is consulted only at bounded decision points. Its distinguishing component is a *Structural Verification Layer*: before an answer is emitted, the draft is decomposed into atomic claims and each is checked against the triples the agent actually traversed. Claims that cannot be grounded trigger targeted re-exploration or are removed, and every answer AGR returns is paired with the supporting triples that justify it.

We are precise about what that layer buys, because the ablation of Section 6 is precise about what it does not. Claim verification does not measurably change how often AGR is right; it changes how often AGR is right *for a reason it can exhibit*. The contribution is the output contract — an answer together with the traversed evidence for it — rather than an accuracy gain, and Section 6 reports the null that establishes this at the same length as the effects it does detect.

On 400 certified questions from each of WebQSP [8] and ComplexWebQuestions [9], over a Freebase-derived environment of 2.59 million entities and 8.31 million triples, AGR reaches Hits@1 of 0.755 and 0.522 against four baselines sharing one frozen backbone and one call budget, and it does so at roughly half the language-model calls of the strongest of them. On ComplexWebQuestions it is the only system whose accuracy rises with hop count. Its advantage over the agentic baseline is located rather than asserted: on the questions where the shared cap lets that baseline finish it is slightly ahead, and the entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it.

Contributions.. This paper contributes:

1. **AGR**, a state-machine agent over a constrained graph tool API whose answers carry the traversed triples that support them, so the reasoning is auditable rather than merely asserted (Section 3).
2. A **Structural Verification Layer** that checks atomic claims against traversed adjacency before emission, with its repair routes and their measured firing rates reported as they behave rather than as designed (Section 3.2).
3. A **controlled comparison** against four baselines on one frozen backbone under one call budget, so that differences are attributable to architecture rather than to model capacity (Section 5).
4. A **component-level ablation** with paired significance testing, which finds that removing the planner *improves* accuracy on the shallower benchmark while reducing token use, and that three of four components show no detectable effect (Section 6).
5. A **hand-read census** of all 259 remaining failures, naming the *echo attractor* — a true, grounded entity one hop from the answer that every grounding check passes — and quantifying label defects in both benchmarks (Section 7).

Section 2 positions AGR against prior work. Section 3 specifies the framework and the verification layer, Section 4 the environment and experimental protocol, and Section 5 the measurements. Section 6 attributes them to components, Section 7 reads what remains, and Section 8 states the limits.

2. Related Work

2.1. Static Retrieval over Graphs

Retrieval-augmented generation [2] attaches a language model to an external corpus by embedding the question, retrieving lexically or semantically similar passages, and letting the model answer from them. Where the answer sits in a single retrievable passage this works well; where it must be composed across passages, the ranking signal is measuring the wrong thing, because the passage carrying the intermediate entity resembles neither the question nor anything the system yet knows to look for. Surveys of the graph–language-model intersection identify this as the central limitation of flat retrieval [10].

Knowledge graphs make the missing structure explicit, storing facts as typed edges between identified entities so that composition becomes a traversal rather than an inference over prose. GraphRAG [4] builds a graph over a corpus and retrieves structured context in place of passages, and related systems extend the idea to long-term memory [11] and temporally-qualified facts [12]. All of them retrieve *statically*: a subgraph is fetched around the linked entities in one shot and handed to the model. Because retrieval runs first and runs once, it cannot know what the second hop will need, and a fixed-radius neighbourhood is either too small to contain the answer or so large that the relevant edges are lost among thousands of irrelevant ones.

2.2. Agentic Navigation

Interleaving retrieval with reasoning removes that constraint. The model takes one step, observes what it found, and decides where to look next;

retrieval becomes navigation and the system becomes an agent operating on the graph rather than a pipeline reading from it. This inherits the tool-use and self-reflection patterns established for language agents generally [13, 14, 15].

Think-on-Graph [5] established the pattern for knowledge graphs with model-guided beam search over relations and entities. Plan-on-Graph [6] added two mechanisms to it: decomposition into sub-objectives, and backtracking driven by reflection. Reasoning-on-Graphs [7] takes a different route, generating grounded relation paths as explicit plans and then traversing them. Others add tool abstractions [16], cope with incomplete graphs [17], or generalise across structured sources [18], and recent surveys treat the convergence as a research programme in its own right [19, 20].

KBQA-o1 [21] is the closest system to AGR's search strategy and the most important to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search — a policy model proposing actions, a reward model scoring them, and rollouts whose values propagate back through the tree — around a ReAct-style agent generating logical forms against Freebase, with the exploration additionally producing annotations for incremental fine-tuning. AGR does not do this. Its search is guided best-first with a bounded beam and backtracking, and it carries neither rollouts nor value backpropagation; the two are separable on that basis, and we name the difference rather than blur it.

2.3. Verification and Self-Correction

A parallel line checks generated content after the fact. Chain-of-Verification [22] has a model draft a response, plan verification questions

about it, answer those independently, and regenerate; multi-agent debate [23] achieves a related effect by having several instances critique one another. Both establish the premise this paper's verification layer rests on — that decomposing a draft and checking its parts reduces factual error — but both check against the model's own knowledge rather than an external symbolic source, and the limits of models correcting themselves without such a source are documented [24]. Hu et al. [25] demonstrate a self-correcting agentic graph pipeline in a clinical setting, and FActScore [26] decomposes generations into atomic facts for evaluation, which is the same decomposition applied as a metric rather than as a control step.

What none of them provides is a claim-level check against the *traversed* triples, performed *before* the answer is emitted, inside a graph-navigating agent. That is the gap Section 3.2 fills.

2.4. The Attribution Gap

A second gap is methodological. Agentic systems are assemblies — decomposition, scoring, backtracking, self-correction — and the literature reports them as wholes. When such a system outperforms a static baseline, the published work rarely establishes which mechanism produced the gain, or whether some contribute nothing. Later work then inherits the entire assembly on faith, along with its cost.

The exception is instructive. KBQA-o1's own ablation removes MCTS while leaving every other component intact and drops GrailQA F1 from 78.5 to 48.5 [21] — a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated rather than reporting the assembly whole. Section 6 applies the same instrument to AGR, and reports the

components for which it detects nothing at the same length as the one for which it does.

3. The AGR Framework

AGR inverts the usual agent pattern. Rather than handing the model a set of tools through native function-calling and letting it drive the loop itself, a program orchestrates every step, calls the graph tools deterministically, and consults the model only at bounded decision points through prompts constrained to return JSON. Three consequences follow directly. Every graph operation is logged by construction rather than reconstructed from the model's account of what it did. Budgets are enforced by counters a router checks, not requested in a prompt the model may ignore. And a malformed model response degrades one decision rather than corrupting control flow: every call is wrapped in a parser that, on failure, spends one metered retry asking the model to repair its own output before falling back. What follows specifies the loop that produces a traversal (Section 3.1), the layer that checks a draft against it before an answer is emitted (Section 3.2), and the budgets that bound the whole loop's cost (Section 3.3).

3.1. State Machine and Tool API

Figure 1 gives the topology: six agent nodes over a shared typed state, with two cycles. The **Planner** decomposes the question into an ordered chain of sub-objectives in one model call; **Explorer** and **Evaluator** form a search loop, expanding the frontier and judging whether the active sub-objective is satisfied; **Backtracker** restores an earlier, higher-scoring frontier and bans the edges that led away from it, which is what turns backtracking into a

search operation rather than a deterministic no-op against an unchanged scorer. Two properties of the topology matter beyond the nodes themselves. It contains cycles rather than running once through, because exploration continues until the evaluator is satisfied and verification can send the agent back to explore. And every path from Evaluator to Answerer passes through the **Verifier** — there is no route that reaches the Answerer without it, which is the structural expression of this paper's contribution and the subject of Section 3.2.

Five graph operations back this loop, specified as parameterised Cypher templates rather than left to free-form generation the model would write itself — a generated-query failure is indistinguishable, in the results, from a reasoning failure, and this paper's central claims are about reasoning and verification, not about syntax. Only four of the five are reachable from a node in the final design. `verify_triple` was built first, to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities; it did not survive contact with generated text, because claim decomposition phrases relations in English ("played for", "is the capital of") that rarely correspond to a single Freebase predicate, and matching on them rejected claims that were true. `verify_connection` replaced it: it drops the relation and asks only whether the two entities are adjacent, directly or through one mediator node, which is the check Section 3.2 actually uses. `verify_triple` remains in the API and in its unit tests, but nothing in the running loop calls it.

Each candidate edge the Explorer considers is scored by

$$\text{score}(c) \;=\; \alpha \cos(v_{\text{rel}(c)},\, v_{\text{obj}}) \;+\; (1-\alpha)\,\text{LLM}(c),$$

blending the cosine similarity of the candidate relation's embedding against

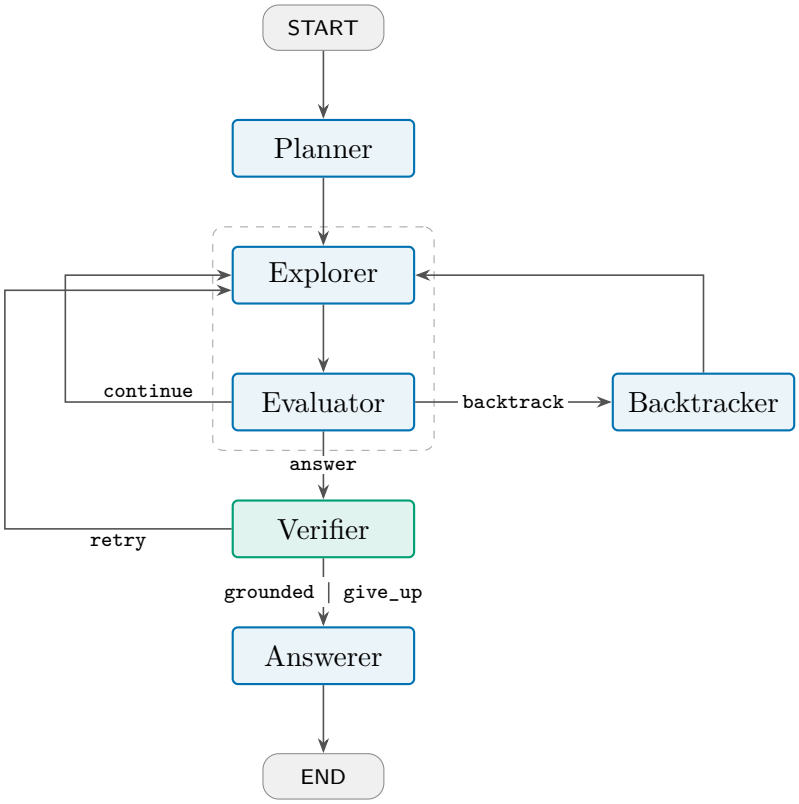

Figure 1: The AGR state machine. The dashed region is the Explorer ↔ Evaluator search loop; Verifier → Explorer is verification-driven re-exploration. Both cycles are bounded by the budgets of Section 3.3. Every path from Evaluator to Answerer passes through the Verifier.

the embedding of the *active sub-objective* — not the original question, which is the concrete mechanism through which decomposition is meant to help — with a batched model judgement over a pre-filtered shortlist of twenty, at the frozen $\alpha = 0.7$. The blend is applied after the model call rather than inside the prompt, so a prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition; setting $\alpha = 1$ disables the

model term entirely, which is the ablation Section 6 reports.

3.2. Structural Verification Layer

The navigation loop above ends with a set of traversed triples and a set of accumulated candidate entities; nothing in it constrains what the final answer *asserts*. The verifier is the check that does. A single model call drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims $\langle h, r, t \rangle$ over entity *names*, since the model has never seen a graph identifier; the relation slot is left as free text, because asking the model to name a Freebase predicate would fail as unreliably as verify_triple did.

Each claim is routed through up to three tests, and a claim leaves at the first one that settles it. **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs; a claim is supported if its pair is in that set, and — uniquely among the three routes — every traversed triple joining the pair is attached as citable evidence. **verify_connection** catches claims whose endpoints were both reached but never directly walked together, querying the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. The order is deliberate rather than incidental: symbols decide and the model is consulted only where the graph is silent, so cheap-to-expensive ordering also means the model adjudicates a residue rather than every claim, and only the first route yields evidence that can be attached to the answer.

The router that follows has three outcomes. No unsupported claim routes

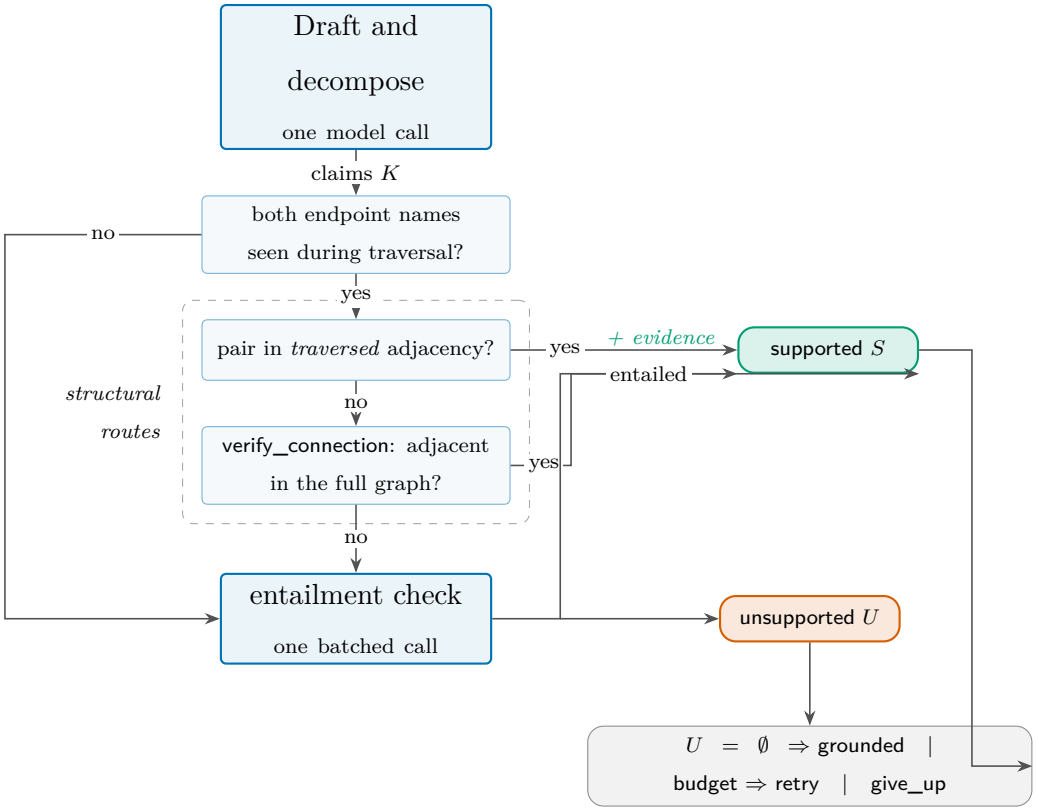

Figure 2: The path one atomic claim takes through the verification layer. Three tests are tried in a fixed, cheap-to-expensive order and a claim leaves at the first that settles it. Only traversed adjacency attaches citable evidence; a claim naming an entity the agent never reached skips both structural routes entirely.

to grounded, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to retry: the verifier re-targets the agent on the first unsupported claim and control returns to the Explorer. Otherwise the router takes give_up, and one rewrite call restates the draft around a *deterministically filtered* entity list — an entity survives if it appears in a supported claim or in no unsupported claim — rather than letting the model regenerate the list itself, which an earlier design did and which reintroduced exactly the ungroundedness the layer exists to prevent.

The layer's reach should be stated plainly here, because it bears directly on how the null result of Section 6 should be read. On the 80-question development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost; the remaining 3 took give_up, and the retry route did not fire even once, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget by the time it got there. On most questions, in other words, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes. Sections 5 and 6 report what it measurably changes at test scale; engineering detail beyond that — defects found along the way and recorded rather than quietly repaired — is in the repository rather than reproduced here.

3.3. Budgets and Termination

Table 1 lists the caps that bound every run. They are enforced in routers and the model client rather than requested in prompts, and three of them — depth, wall-clock, and verification iterations — are checked at two separate

Table 1: Budget configuration. Every row is hashed into the run configuration, so a reported result can always be traced to the caps that produced it.

Parameter	**Value**	**Enforced in**
max_depth	4	post-evaluation *and* post-verification routers
beam_width	3	Explorer (per-anchor slice)
max_anchors	5	Explorer *and* Evaluator (frontier slice)
max_backtracks	3	post-evaluation router
max_llm_calls	25	model client, before every call
max_verify_iters	2	post-verification router *and* the verifier
max_seconds	300	post-evaluation *and* post-verification routers

sites each, because the retry edge re-enters the Explorer without passing back through the router that would otherwise catch it; closing that gap is what lets Table 1 be called a guarantee rather than a convention.

Every cycle in Figure 1 therefore passes through a router that checks or decrements a monotone counter, so no sequence of model outputs, however adversarial, can produce a non-terminating run. The cost of enforcing this is small in the common case: the grounded path above adds nothing beyond the single drafting call already required to produce a draft. Section 5.2 reports what the layer costs in aggregate, on the questions where it is not a pass-through.

4. Experimental Setup

4.1. Knowledge Environment

All five systems navigate one shared environment, a property graph of 2.59 million entities and 8.31 million triples over 7,058 distinct relation types, built as the union of the per-question subgraphs distributed with Reasoning-on-Graphs [7] and derived from Freebase [3]. Just under two-thirds of the nodes — 63.7% — are unnamed mediators encoding n-ary facts, which is why the tool layer traverses them transparently rather than exposing them as candidate entities (Section 3).

An environment assembled this way is friendlier than full Freebase, and the paper states the consequence rather than leaving it to be inferred. Before any system ran, a validation gate measured what fraction of questions have at least one gold answer reachable from a topic entity within the four-hop cap: 97.0% on the WebQSP sample and 99.2% on the ComplexWebQuestions sample. That ceiling is the reason a failure in Section 7 can be attributed to the system rather than to a missing edge — almost every question is answerable in principle, so a wrong answer is a navigation or selection error, not an absent fact. It is equally the reason **absolute accuracies here do not transfer to a full-Freebase setting**. Every comparison in Section 5 is between systems traversing the identical environment, which is what the comparison is for; none of the numbers should be read against published figures obtained over a different substrate.

4.2. Benchmarks, Baselines, and Backbone

We evaluate on WebQSP [8] and ComplexWebQuestions [9], the two standard multi-hop KGQA benchmarks over Freebase. **We sample** 400 **questions from each** rather than running the full 1,628 and 3,531 splits — a pre-specified fallback adopted when full-split cost was projected at roughly three times the budget available for the work. The sample is stratified by hop count, and the resulting 95% bootstrap intervals are roughly ± 4 to ± 5 points on Hits@1; Section 8 says which of the paper's claims survive that width and which are reported as directions rather than effects.

Four baselines span the design space, each a paradigm rather than a tuned competitor. A **no-retrieval** control answers from parametric memory alone, establishing what the backbone knows without the graph. **Vector-RAG** retrieves verbalised triples by embedding similarity. **Static GraphRAG** [4] fetches a fixed-radius neighbourhood around the linked topic entities in one shot. **Think-on-Graph** [5] is the agentic comparator, performing model-guided beam search over relations and entities, and is the system AGR is measured against throughout. Baselines were debugged on development data and certified before any test question ran, against a pre-specified diagnostic targeting the one failure that would be an implementation defect rather than a paradigm limit: retrieval found the gold answer and the system hedged anyway.

Every system shares one frozen backbone — gpt-5.4-mini, at temperature 0.0 with reasoning effort disabled — together with the entity resolver, the question files, the output schema, and *one budget meter with the same* 25*-call cap.* Both graph-navigating systems seed from the datasets' annotated topic

entities, which removes entity linking as a confound. Holding all of this constant is what licenses the paper's central inference: a difference between two systems is attributable to architecture, not to model capacity, retrieval corpus, or spend. Two things are deliberately not held constant and are named here rather than buried — the baselines receive a plain answering prompt while AGR's drafting prompt carries the grounding provisions that are part of the system under test, and the two navigators consider candidate sets of different widths, 40 relations and 20 neighbours for Think-on-Graph against 300 and 200 for AGR. The second is where a reader should look first for a confound, and Section 5.2 reports the measurement that bounds it.

4.3. Metrics and Protocol

Accuracy is reported as **Hits@1** and per-question **F1**, the latter because a system can score a clean hit on set-intersection while asserting several wrong entities alongside the right one. A **hedge** is an empty predicted entity set; hedges score zero on both metrics and hedge rate is reported as its own column, so a system buying precision through abstention is visible rather than flattered. Hits, hedges, and wrong answers always partition the whole set: nothing is dropped from the denominator, including questions the environment provably cannot answer.

Groundedness is measured at two levels. The structural tier is deterministic and applies to every asserted entity in all 4,000 records: an entity is grounded if and only if it exists as a node in the environment and connects to a topic entity within four hops. The semantic tier asks the harder question — whether the entity is supported *as the answer* — by retrieving the graph's own shortest connecting paths for a stratified sample

of 60 answers per system per dataset and having a language model judge whether any path expresses the relationship the question asks about, under the governing instruction that mere connectedness is not support. That judge was validated against a human annotator on 100 items, reaching 85% agreement and Cohen's $\kappa = 0.70$. Because path retrieval takes only the three shortest paths, semantic-tier rates are conservative lower bounds, depressed uniformly across systems and read as a between-system comparison rather than as absolute support rates. Efficiency is tokens and model calls, both exact from the shared meter.

The protocol was **pre-specified**: research questions, metrics, baseline certification thresholds, and ablation conditions were fixed and documented before the test sets were constructed, and no hyperparameter was tuned on test. The blend $\alpha = 0.7$ and the backtracking threshold $\tau = 0.20$ were selected on a disjoint 80-question development set. Nothing was filed with a public registry, so the claim is pre-specification and not pre-registration. Deviations encountered during execution — and there were several — are recorded in the repository rather than narrated here.

5. Results

5.1. Accuracy

Table 2 gives the main matrix. AGR leads every accuracy cell on both datasets, reaching Hits@1 of 0.755 on WebQSP against the agentic baseline's 0.660, and 0.522 on ComplexWebQuestions against 0.435. The margin over Think-on-Graph is 9.5 and 8.7 points respectively, and the per-question F1 gap is wider still — 0.642 against 0.477, and 0.469 against 0.378 — because

Table 2: Main results, 400 questions per dataset, one frozen backbone and one 25-call budget for every system. Hedge is the proportion of questions answered with an empty entity set.

	WebQSP				**ComplexWebQuestions**			
System	Hits@1	F1	Hedge	Calls	Hits@1	F1	Hedge	Calls
no-retrieval	0.453	0.305	12.2%	1.0	0.307	0.279	38.0%	1.0
Vector-RAG	0.568	0.432	27.0%	1.0	0.203	0.168	48.2%	1.0
GraphRAG	0.340	0.262	55.8%	1.0	0.205	0.183	60.8%	1.0
Think-on-Graph	0.660	0.477	18.8%	12.8	0.435	0.378	22.5%	18.2
AGR	**0.755**	**0.642**	8.2%	6.2	**0.522**	**0.469**	23.0%	8.9

F1 penalises the extra wrong entities a set-intersection hit forgives. All eight paired comparisons against the four baselines reject at the conventional level.

Two features of this table are worth naming because they cut against a simple reading. The static graph baseline is the *weakest* system on both datasets, below even the no-retrieval control — a fixed-radius neighbourhood is not merely insufficient for multi-hop questions, it is actively worse than parametric memory, because it floods the context with edges that are structurally adjacent and semantically irrelevant. And AGR does not lead on every column: on ComplexWebQuestions its hedge rate of 23.0% is marginally *above* Think-on-Graph's 22.5%, so it declines to answer slightly more often there. The accuracy lead is real and it is not bought by answering more freely.

5.2. Cost and the Location of the Margin

AGR reaches those numbers at roughly half the model calls of the agentic baseline — 6.2 against 12.8 on WebQSP, 8.9 against 18.2 on ComplexWeb-

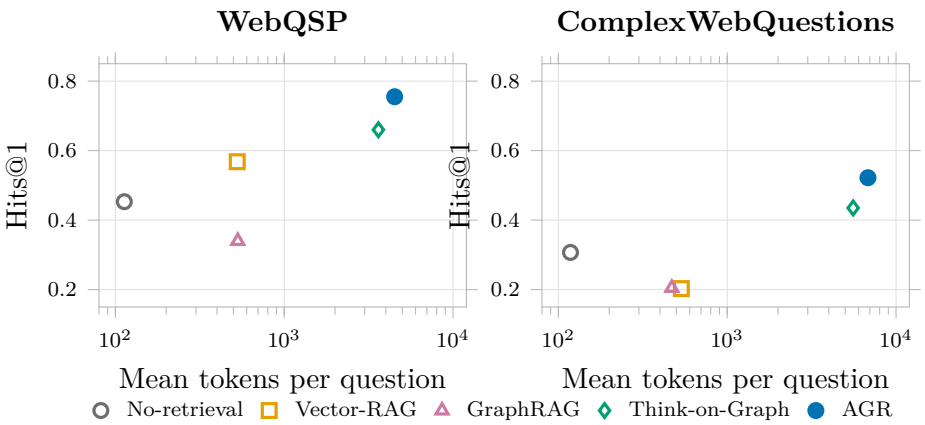

Figure 3: Accuracy against mean tokens per question, both datasets. Cost is plotted on a log scale. The single-call baselines cluster at the left; the two navigators sit at the right, with AGR above and to the left of Think-on-Graph on both panels.

Questions, a ratio of 0.48 on both. Figure 3 plots accuracy against cost for all five systems.

The call saving is genuine and the token saving is not: AGR spends 4,511 mean tokens per question on WebQSP against Think-on-Graph's 3,615, so it is the *number of round trips* that halves, not the volume of text. Fewer, larger calls is the honest description.

The more informative result is *where* the accuracy margin comes from, and it is not where a reader would assume. Splitting the questions by whether the shared 25-call cap cut Think-on-Graph off before it finished reverses the comparison on one side:

On the questions where the budget lets Think-on-Graph run to completion, it is *ahead* of AGR — 0.852 against 0.788 on WebQSP, 0.630 against 0.607 on ComplexWebQuestions. The entire aggregate margin comes from

Table 3: AGR against Think-on-Graph, split on whether the shared call cap truncated the baseline. Read from the per-record budget_exhausted trace flag.

Subset	n	**Think-on-Graph Hits@1**	**AGR Hits@1**
WebQSP			
Think-on-Graph finished	283	**0.852**	0.788
Think-on-Graph clipped	117	0.197	**0.675**
ComplexWebQuestions			
Think-on-Graph finished	224	**0.630**	0.607
Think-on-Graph clipped	176	0.188	**0.415**

the 29% and 44% of questions on which the cap truncates it, where its accuracy collapses to 0.197 and 0.188 while AGR still reaches 0.675 and 0.415. AGR's own call budget never binds: across all 800 test questions it hit the 25-call cap zero times.

The correct reading of this is affordability, not resolution quality. Exhaustive beam search finds answers well when it is allowed to finish; guided search finds them almost as well and finishes within a budget the beam search cannot. On a fixed spend the second is the better architecture, and the first would win if spend were unconstrained — a claim about operating regimes rather than about reasoning power, and narrower than the aggregate table alone would suggest.

5.3. Accuracy by Hop Depth

Figure 4 stratifies by the number of hops separating a topic entity from the gold answer.

On ComplexWebQuestions, AGR is the only system whose accuracy *rises* with hop count: 0.46 at one hop, 0.55 at two, 0.57 at three or more. Every

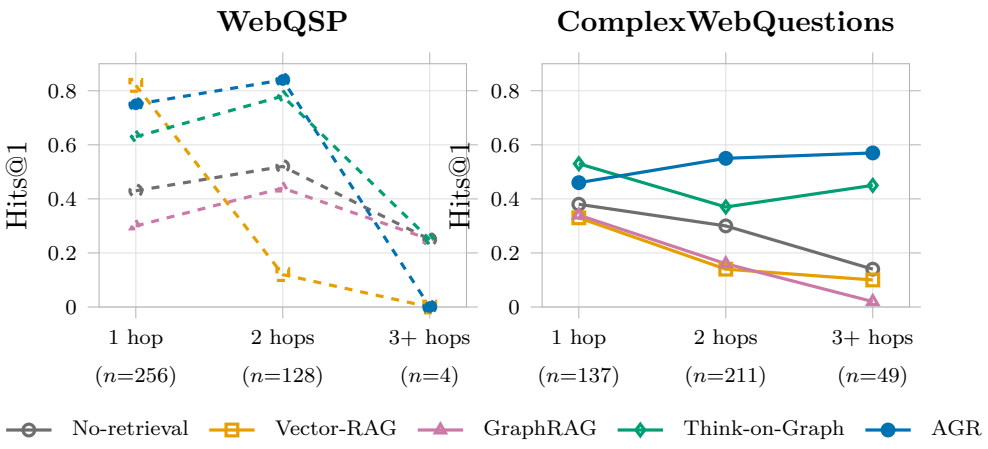

Figure 4: Hits@1 by hop stratum. On ComplexWebQuestions (right) AGR is the only system whose accuracy rises monotonically with hop count. The WebQSP three-hop stratum holds four questions and no weight should be placed on it.

other system ends below where it started. That phrasing is deliberate and the weaker alternative is the true one — Think-on-Graph does not decay monotonically, it falls from 0.53 to 0.37 and then partly recovers to 0.45, which is a different shape from a steady decline and is reported as such. The static baselines do collapse outright, GraphRAG from 0.34 to 0.02.

A rising curve is the signature the architecture predicts. Decomposition converts one hard question into several easy ones, so the questions that punish single-shot retrieval hardest are the ones an agent that re-plans between hops should handle best. The WebQSP panel carries no comparable evidence: its three-hop stratum contains four questions, which is why the claim is scoped to one dataset.

5.4. Groundedness

Across both datasets AGR asserted 1,709 entities and **zero of them were ungrounded** — every entity it named exists in the environment and is reachable from a topic entity within the hop cap. The parametric control asserted 1,001 entities of which 22.1% failed that test.

This result belongs to graph navigation, not to the verification layer, and the distinction is load-bearing. Think-on-Graph also reaches exactly zero ungrounded assertions, on 1,265 entities, without any claim-checking component whatsoever. Whatever the verification layer contributes, it is not this number. What the comparison establishes is that an agent which can only name entities it retrieved cannot fabricate one, and that this property follows from the retrieval discipline rather than from any check applied afterwards — a conclusion that costs this paper its most quotable claim for the component it contributes, and is the conclusion the measurement supports.

The semantic tier separates the systems where the structural tier cannot. Judging whether an asserted entity is supported *as the answer*, AGR reaches 66.7% and 48.3% against Think-on-Graph's 61.7% and 43.3%, with the parametric control at 63.3% and 36.7%. These are conservative lower bounds by construction (Section 4.3) and the gaps are small relative to the sample of 60 per cell, so they are reported as consistent with the accuracy ordering rather than as independent confirmation of it.

Table 4: Component ablations. ΔF1 is ablated minus reference: positive means the system got better without the component. p from McNemar's test on paired per-question correctness.

	WebQSP ($n = 200$)			**CWQ** ($n = 198$)		
Removed	ΔF1	p	Tokens	ΔF1	p	Tokens
Planner	+0.083	**0.006**	−31%	−0.047	0.088	−21%
Backtracking	−0.013	0.727	−7%	0.000	1.000	−18%
Verification	+0.004	1.000	−2%	−0.009	1.000	−6%
Model scoring	−0.016	0.664	−25%	−0.008	0.481	−31%

6. Component-Level Attribution

Agentic systems are assemblies, and the literature reports them whole. This section takes AGR apart. Four conditions each remove exactly one component from the unmodified system and re-run the identical questions: no planner, no backtracking, no claim verification, and embedding-only scoring ($\alpha = 1$, disabling the model term in the edge scorer). Each condition is compared against the reference on the same half-split — $n = 200$ on WebQSP, $n = 198$ on ComplexWebQuestions — with McNemar's test on the paired per-question outcomes.

Throughout this section a delta is ablated minus reference, so a positive number means removing the component *improved* the metric. Table 4 gives all four.

6.1. When Planning Hurts

One component produced a detectable effect, and it produced it in the wrong direction. **Removing the planner improves WebQSP F1 by**

0.083 ($p = 0.006$) while cutting token use by 31%. Twenty-one questions were answered correctly only without the planner against six correct only with it — the disagreement is not marginal, and the cheaper system is the more accurate one.

The mechanism is over-decomposition. WebQSP is predominantly a single-hop benchmark: 256 of its 400 questions need one hop. Asked to decompose a question that has no compositional structure, the planner manufactures sub-objectives that are not really separable, and the explorer then scores candidate edges against a fragment of the question rather than against the question — discarding exactly the context that would have identified the right relation. The planner prompt carries a single-hop exemplar precisely to demonstrate that not decomposing is a valid output, and it is evidently not enough.

On ComplexWebQuestions the direction reverses, as the mechanism predicts: that benchmark is genuinely compositional, only 137 of 400 questions are single-hop, and removing the planner costs 0.047 F1. **That arm does not reach significance** ($p = 0.088$) and is reported as a direction rather than an effect. It is not described as marginally significant, because at this sample size that phrase would assert more than the data carries.

Pooling the two datasets would average a significant improvement against a non-significant harm and report approximately nothing. The asymmetry *is* the result: decomposition is not a general-purpose benefit but a mechanism whose value depends on whether the question has compositional structure to find. The actionable consequence follows directly, and it is the recommendation this paper would most like to see adopted — **gate decomposition**

on question structure rather than applying it unconditionally, predicting the number of hops from the question and invoking the planner only above a threshold. On these benchmarks that policy would capture the WebQSP improvement and the ComplexWebQuestions capability at once.

6.2. Three Components with No Detected Effect

The remaining three conditions moved nothing detectable, and they are reported at the same length as the one that did. A field that reports only what moved cannot attribute anything.

Backtracking. Removing it changes WebQSP F1 by -0.013 ($p = 0.727$) and ComplexWebQuestions F1 by 0.000 ($p = 1.000$), while saving 7% and 18% of tokens. The mechanism fires rarely — 38 and 108 backtracks across the half-splits — and the backtrack budget refuses further attempts on only 6.2% of questions. A component this seldom exercised cannot show a large effect, and the honest summary is that the search rarely needs to undo a decision, not that undoing is worthless.

Model scoring. Setting $\alpha = 1$ removes the language model from edge selection entirely, leaving pure embedding similarity. F1 falls by 0.016 and 0.008, neither significant, while tokens drop 25% and 31%. The model term is the single most expensive part of the navigation loop, and this experiment cannot establish that it earns its cost. One confound is recorded rather than smoothed over: at $\alpha = 1$ the scorer takes an early return that leaves the backtracking trigger's signal maximum computed over a smaller candidate set, so this condition's elevated backtrack count is not attributable to the scoring change alone.

Claim verification. Removing the layer this paper contributes changes

WebQSP F1 by +0.004 and ComplexWebQuestions F1 by −0.009, $p = 1.000$ on both. Across the half-splits — 200 questions on WebQSP and 198 on ComplexWebQuestions — the two conditions disagreed on exactly one question each. **Claim verification does not measurably change how often AGR is right.**

The result is expected rather than disappointing, for the reason Section 3.2 gave before any of these numbers appeared: on most questions the layer is a pass-through certifying a draft that was already grounded. On test it asks for a repair on 52 of 800 questions, exploration actually resumes on 28, and 13 end grounded that did not start so. A mechanism that alters the output on a few per cent of questions cannot move an aggregate accuracy metric, and an ablation is the right instrument for establishing that rather than assuming it either way.

What the layer delivers is therefore not an accuracy gain but the output contract: an answer paired with the traversed triples that support it, and an entity list that is a deterministic function of the verdicts rather than a second generation. That is an auditability claim, it is what Section 5.4 shows the layer cannot claim credit for on groundedness, and it is the claim the measurements support.

Power

These are nulls at low power, not demonstrations of absence, and the arithmetic is worth stating exactly. McNemar's exact test sees only the *discordant* pairs — the questions on which the two conditions disagree — and the null conditions produced very few of them.

Removing claim verification changed the outcome on exactly one question

per dataset. With a single discordant pair no split whatsoever can reach significance, so that row is not a test that failed to reject; it is the two conditions agreeing on 396 of the 398 paired questions, which is a statement about how rarely the layer alters an answer rather than about statistical power. Backtracking produced 8 and 11 discordant pairs, and model scoring 21 and 18. At those counts the smallest imbalance the exact test could have called significant is 8 and 9 questions for backtracking — all eight would have had to fall the same way — and 11 and 10 for model scoring, which is between 4.0 and 5.5 points of accuracy. Differences below that were undetectable at any power, whatever their true size.

For calibration: detecting a true 2:1 discordance ratio at 80% power requires roughly 72 discordant pairs, and the largest of these conditions produced 21. At that count the smallest ratio detectable at 80% power is nearer 4:1 — a true 2:1 effect would have been caught about a quarter of the time. **Three of four components show no effect at this power**; none is shown to have no effect.

7. Error Analysis

Every one of AGR's 259 remaining failures — 86 on WebQSP and 173 on ComplexWebQuestions — was read and labelled by hand against its full execution trace, separating wrong answers from hedges throughout because the two are not the same kind of failure: a wrong answer is a reasoning error, a hedge is usually a coverage gap that never produced a committal claim. Figure 5 gives the census.

The bulk of the distribution is unsurprising — choosing the wrong relation

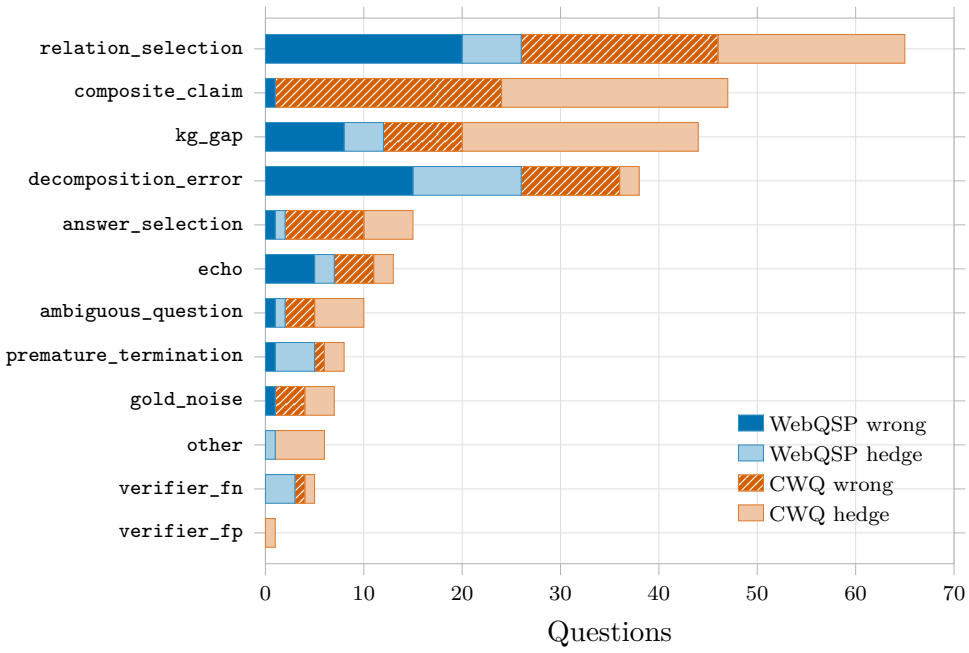

Figure 5: Hand-read census of all 259 residual failures, by category, with wrong answers and hedges kept separate. Relation selection and composite claims dominate; the two categories this section discusses are not the largest, but they are the two whose lessons transfer beyond this system.

to follow, and composite claims whose unverifiable component drags down a verifiable one, together account for most of the residue. Two smaller categories are reported here instead, because they generalise past this architecture.

7.1. The Echo Attractor

The most instructive failure mode is one that every grounding check in this paper passes. The system resolves a real entity, reached through a real edge, and states a fact that is genuinely true — and it is not what was asked.

We name this the **echo attractor**: a true, well-grounded entity sitting one hop from the answer in the graph's structure.

It takes two shapes. *Topic confusion* picks a neighbour in a tight naming cluster: asked who plays Lex Luthor on *Smallville*, the system answers that John Glover plays Lionel Luthor — a different character in the same fictional family, with three further Luthor variants surfacing in the trace before the run gives up. *Granularity* picks the right entity family at the wrong level: asked for a country, the system answers "Kingdom of Denmark" where the gold answer is "Denmark" — a genuinely distinct node, the formal sovereign-state entity rather than the common-name one.

What makes this a mechanism rather than a residue category is that nothing went wrong in the parts of the system built to prevent things going wrong. The right edge exists, is retrieved, and is walked; the entity returned is verifiably present in the environment; every structural check certifies it. The failure is entirely in *which* true fact gets surfaced, which is architecturally distinct both from following a wrong edge and from the fact being absent. A claim-level check against traversed adjacency cannot catch it by construction — the claim is true.

The consequence for evaluation methodology is larger than the consequence for this system. Because the attractor is a property of the graph's neighbourhood structure rather than of any one search policy, architecturally unrelated systems converge on the *same* wrong entity for the *same* questions. Any protocol that treats systems as independent observers therefore cannot see it: consensus filtering, ensembling, and majority-vote gold repair all take agreement as evidence of correctness, and here agree-

ment is evidence of a shared structural bias instead. Such a protocol will confidently ratify the wrong answer in exactly the cases it was introduced to catch. We observed this directly in the pass described next, where five systems agreeing on one answer turned out far more often to be five systems echoing than to be a bad label.

The echo attractor accounts for 13 of the 259 failures. The rate is small and the sample does not support a precise one; the contribution is the named mechanism and its methodological consequence, not its frequency at this n.

7.2. Benchmark Label Defects

The census required a second pass, because a failure against a wrong gold label is not a failure of the system. A consensus pre-pass flagged every question where systems agreed on an answer the benchmark scored wrong, and each flagged question was then adjudicated individually against the graph and the question text.

The two counts must be kept apart. The pre-pass **flagged** 58 WebQSP and 47 ComplexWebQuestions questions; adjudication **confirmed** a genuine label defect in 22 and 19 of them respectively — 5.5% and 4.8% of each sample, and 57 distinct questions once the one question appearing in both counts is resolved. Confirmed defects split into two kinds: gold answers that are simply wrong (12 and 17), and questions whose phrasing admits several defensible answers of which the benchmark lists one (10 and 2).

The gap between flagged and confirmed is not adjudication noise — it is the echo attractor, measured. Roughly half of what the consensus pass flagged was not a bad label at all but several systems converging on the same structurally-adjacent wrong entity, which is precisely the failure

mode the previous subsection predicts a consensus protocol will misclassify. Had we accepted the pre-pass without per-item adjudication, we would have "corrected" the benchmark toward the systems' shared bias and reported an inflated accuracy for every system including our own.

All results in Section 5 are reported over the full samples, without excluding the confirmed defects. Excluding them raises every system's accuracy by roughly the defect rate and changes no ordering; we report the unadjusted figures as the primary result and note that the 400-question samples carry a label-defect floor of about five per cent, which bounds what any system evaluated on them can be measured to achieve.

8. Discussion and Limitations

The limits of this study bound its claims in ways worth stating plainly.

Sample size. Every result rests on 400 questions per dataset rather than the full splits, giving 95% bootstrap intervals of roughly ± 4 to ± 5 points on Hits@1. The main accuracy margins clear that width comfortably — AGR's 9.5-point lead over the agentic baseline on WebQSP and 8.7 on ComplexWebQuestions — as does the planner effect, which is a paired test on per-question outcomes rather than a comparison of two intervals. The groundedness contrast, at zero against 22.1%, is not close. What the sample does not support is fine distinctions: the semantic-tier gaps of a few points in Section 5.4, and the hedge-rate difference on ComplexWebQuestions, are within noise and are reported as such.

One backbone. All five systems run on gpt-5.4-mini, frozen and closed. This paper establishes how these architectures compare *on one backbone*,

not how the comparison scales across model capacities. A larger model might close the gap between guided and exhaustive search by finishing more questions inside the same budget, or widen it; nothing here decides which.

Nondeterminism. Temperature-zero decoding against a hosted API is greedy, not deterministic, and measured trajectory stability is about 67%. Each system was run once. No seeds were averaged, and a reader should not assume they were; run-to-run variation is a real component of the differences reported, bounded but not eliminated by the paired designs used for the significance tests.

The environment. Per-question subgraphs with a 97%-plus answer-reachability ceiling are a friendlier substrate than full Freebase. Between-system comparisons hold, because all five systems navigate the identical graph; absolute accuracies do not transfer.

Ablation power. The four ablations run on half-splits of $n = 200$ and $n = 198$, and the paired test sees only the questions on which two conditions disagree — as few as one, for claim verification. Three of four components show *no effect detected at this power*, which is not the same as no effect; Section 6.2 gives the smallest difference each condition could have detected, between 4.0 and 5.5 accuracy points where a test was possible at all.

Scope. English factoid questions over a static graph, with no temporal facts, no graph construction, and an offline prototype. Latency figures are a cost measure, not a production-readiness claim.

The most actionable finding points forward rather than back. If decomposition helps only when a question has compositional structure — improving accuracy by 0.083 F1 where it does not exist, and costing 0.047 where it does

— then the planner should not run unconditionally. A hop-count predictor gating the planner would capture both arms of that asymmetry, and it can be evaluated with the ablation instrument used here. More generally, the result suggests that agentic components should be treated as conditionally rather than universally beneficial, and that the condition is worth measuring.

9. Conclusion

We presented AGR, a knowledge-graph question-answering framework built as an explicit state machine over a constrained graph tool API, whose answers are paired with the traversed triples that support them. Against four baselines sharing one frozen backbone and one call budget, it leads every accuracy cell on both benchmarks at roughly half the model calls of the strongest of them, and it is the only system whose accuracy rises with hop count on the compositional benchmark. Its advantage is located rather than asserted: on the questions where the shared budget lets the agentic baseline finish, that baseline is slightly ahead, and the entire aggregate margin comes from the questions where the budget truncates it. The finding is about affordability under a fixed spend, and we report it as such.

The result we consider most useful is the one that did not go our way. Agentic systems are assemblies, and the field reports them whole. We took this one apart, and three of its four components could not be shown to earn their cost — including the claim-verification layer this work set out to contribute, which does not measurably change how often the system is right. It changes how often the system is right for a reason it can exhibit, which is an auditability claim and not an accuracy one. The fourth component was

actively harmful on the shallower benchmark: removing the planner improved accuracy and cut token use, because decomposing a question that has no compositional structure discards the context that would have answered it.

That pattern is a claim about how this field measures rather than about this architecture. A component adopted because the assembly containing it performed well may be contributing nothing, or less than nothing, and the only way to know is to remove it and test. We would rather report three nulls and a harmful component honestly than a system that works for reasons nobody has checked.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported here.

Data availability

Code, prompts, per-question run records, and the analysis scripts that produce every figure and table in this paper are available at `https://github.com/sakif-khan/agentic-graph-reasoning` under the MIT licence. The knowledge environment is derived from Freebase [3] and is rebuilt from its public source by the scripts in that repository rather than redistributed; the question sets are the public WebQSP [8] and ComplexWebQuestions [9] releases.

References

[1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Transactions on Information Systems 43 (2) (2025) 1–55.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: A collaboratively created graph database for structuring human knowledge, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, 2008, pp. 1247–1250.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv preprint arXiv:2404.16130 (2024).

[5] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2307.07697.

[6] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, arXiv:2410.23875.

[7] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.01061.

[8] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, J. Suh, The value of semantic parse labeling for knowledge base question answering, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2016, pp. 201–206.

[9] A. Talmor, J. Berant, The web as a knowledge-base for answering complex questions, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 641–651.

[10] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, J. X. Yu, A survey of graph meets large language model: Progress and future directions, arXiv preprint arXiv:2311.12399 (2023).

[11] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, pp. 59532–59569.

[12] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, J. Liu, T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval, in: Proceedings of the 33rd ACM International Conference on Multimedia, 2025, pp. 11880–11889.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[16] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, J.-R. Wen, KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph, arXiv preprint arXiv:2402.11163 (2024).

[17] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, J. Zhao, Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, arXiv:2404.14741.

[18] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, J.-R. Wen, StructGPT: A general framework for large language models to reason over structured data, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, arXiv:2305.09645.

[19] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599.

[20] C. Ma, Y. Chen, T. Wu, A. Khan, H. Wang, Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities, in: Proceedings of the 28th International Conference on Extending Database Technology (EDBT), 2025, pp. 1174–1177.

[21] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan, KBQA-o1: Agentic knowledge base question answering with monte carlo tree search, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2501.18922.

[22] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, J. Weston, Chain-of-verification reduces hallucination in large language models, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 3563–3578.

[23] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning, 2024.

[24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, Large language models cannot self-correct reasoning yet, in: International Conference on Learning Representations (ICLR), 2024.

[25] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, F. Fang, A self-correcting agentic graph RAG for clinical decision support in hepatology, Frontiers in Medicine 12 (2025) 1716327.

[26] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100.